Text-Guided Remote Sensing Super-Resolution Network with Multi-Stage Tail Refinement

Hao Yang, Man-On Pun, *Senior Member, IEEE*, and Xianping Ma

***Abstract*—This study introduces a Text-Guided Remote Sensing Super-Resolution Network with Multi-Stage Tail Refinement (TG-RST), which leverages textual and semantic guidance to enhance feature aligning in remote sensing super-resolution tasks. This is the preview version.**

***Index Terms*—Remote sensing image, super-resolution, multi-modal learning, semi-supervise**

I. Proposed Method

A. Overview

As previously noted, many RSSR methods exhibit consistent residuals between the super-resolved (SR) images and the high-resolution (HR) ground truth. Moreover, these Residuals tend to be highly similar across different samples and are generally easier to predict than the SR images themselves. Based on this observation, this study introduces a Tail model designed to predict the Residual. The final output of the proposed approach is obtained by summing the original SR output from a Tailless RSSR method with the predicted Residual generated by the Residual Tail. Let the HR image be denoted by $\mathbf{x}$, the LR image by $\mathbf{y}$, and the original SR image by $\hat{\mathbf{x}}_0 = \mathrm{SR}(\mathbf{y})$. The Residual can then be expressed as

$$\mathbf{R} = \mathbf{x} - \hat{\mathbf{x}}_0 = \mathbf{x} - \mathrm{SR}(\mathbf{y}). \tag{1}$$

In this work, the Tail model (denoted as $\mathrm{ResTail}(\cdot)$) predicts the Residual directly from the LR image, resulting in

$$\hat{\mathbf{R}} = \mathrm{ResTail}(\mathbf{y}), \tag{2}$$

and the final predicted SR image is computed as

$$\hat{\mathbf{x}} = \hat{\mathbf{x}}_0 + \hat{\mathbf{R}} = \mathrm{SR}(\mathbf{y}) + \mathrm{ResTail}(\mathbf{y}). \tag{3}$$

To realize the aforementioned concepts, this study proposes the Text-Guided Remote Sensing Super-Resolution Network with Multi-Stage Tail Refinement (TG-RST), which aims to enhance RSSR performance by incorporating text captions and a semi-supervised fine-tuning strategy. As illustrated in Figure 1, the proposed TG-RST consists of four stages. In Stage 1 (Sec. I-B), a Tail model (SR Tail) is trained on a large-scale common object (CO) dataset to generate SR images from LR inputs, as shown in Figure 1a. In Stage 2 (Sec. I-C), the SR Tail is used to produce SR images on the same dataset, and the Residual between the HR ground truth and the SR output is computed. A second Tail model (Residual Tail) is then trained to predict this Residual, as shown in Figure 1b. In Stage 3 (Sec. I-D), the Tailless Text-Guided RSSR network is trained on an RSSR dataset to predict SR images (referred to as Plain SR) from LR inputs, as depicted in Figure 1c. Finally, in Stage 4 (Sec. I-E), the Text-Guided RSSR network generates Plain SR images on the RSSR dataset, while the Residual Tail, pretrained in Stage 2, is fine-tuned on the RSSR dataset to predict the Residual between the Plain SR and the RS image ground truth, as shown in Figure 1d. The final output of TG-RST is obtained by summing the Plain SR and the Predicted Residual generated in Stage 4.

B. Stage 1: Pretraining the SR Tail on a Large-Scale Object Dataset

To preserve the overall efficiency of TG-RST and fully exploit the similarity of Residuals in CO data, we introduce a lightweight Tail model that can be efficiently trained on large-scale CO datasets and easily fine-tuned on RS datasets. The architecture of the Tail model is shown in Figure 2. It is designed to sit after a main super-resolution (SR) backbone and refine the output in the HR space with minimal overhead.

The Tail comprises a minimal encoder with a single convolutional layer and a decoder formed by another convolutional layer. The extracted features are iteratively refined and upsampled before producing the final output image. We introduce iterative lightweight SR units, termed Depthwise Separable Blocks (DSBlocks), which combine depth-wise and pixel-wise convolutions with a highly efficient EMA-inspired attention mechanism. This design enables effective feature refinement while avoiding quadratic computational complexity. The detailed architecture of the DSBlock is depicted in the right of Figure 2, and the upsampling process within the Tail is illustrated on the left side of Figure 2.

DSBlocks are employed as iterative feature refinement modules in both LR and HR spaces. Their residual design provides strong scalability and stability during training. Let the initial LR features be defined as

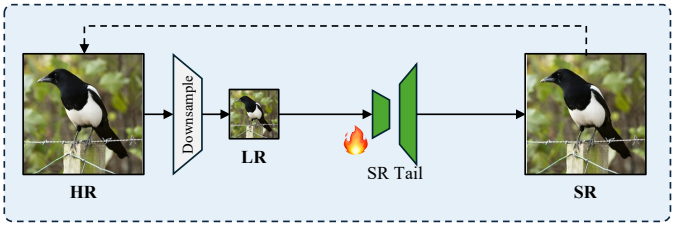
$$\mathbf{F}_{\mathrm{LR}}^{0} = \mathrm{GELU}(\mathrm{ConvEncoder}(\mathbf{y})), \tag{4}$$

where ConvEncoder represents a convolutional encoder layer, GELU is the Gaussian Error Linear Unit activation function, and $\mathbf{y} \in \mathbb{R}^{C\times h\times w}$ denotes the LR input.

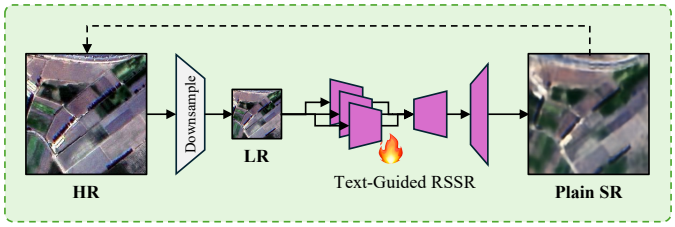
This work was supported in part by the National Natural Science Foundation of China. This is the preview version. *(Corresponding authors: Man-On Pun and Xianping Ma)*

Hao Yang and Man-On Pun are with the School of Science and Engineering, The Chinese University of Hong Kong, Shenzhen, Shenzhen 518172, China (emails: haoyang.official@gmail.com; SimonPun@cuhk.edu.cn).

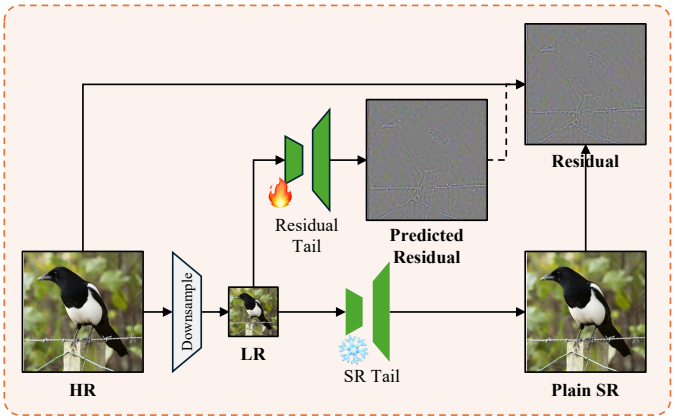
Xianping Ma is with the Faculty of Geosciences and Engineering, Southwest Jiaotong University, Chengdu 611756, China (email: mxp@swjtu.edu.cn).



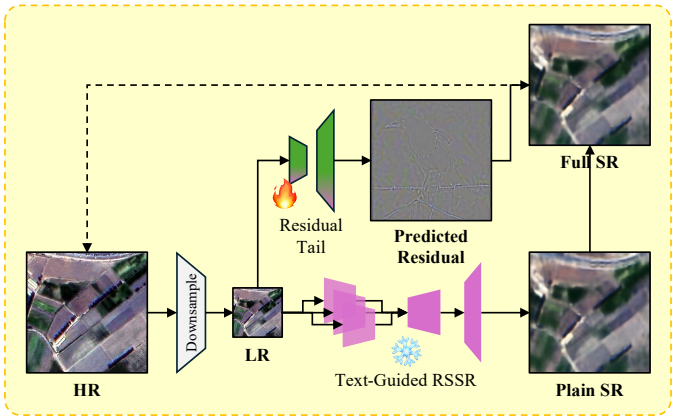
(a) An SR Tail is trained on a large dataset of common objects to generate super-resolved images.



(c) The Text-Guided RSSR method is trained on an RSSR dataset to generate super-resolved images.



(b) The SR Tail is frozen to generate residuals for common objects, while a Residual Tail is trained to predict these residuals.



(d) The Text-Guided RSSR method is frozen to generate plain super-resolved results, and the pretrained Residual Tail is fine-tuned to predict the residuals needed to complete the full super-resolved image.

Fig. 1. The four-step pipeline of the proposed Text-Guided Remote Sensing Super-Resolution Network with Multi-Stage Tail Refinement (TG-RST) is illustrated in panels (a) through (d). During training, the Text-Guided RSSR module and the Tails are marked with a fire icon when active and a snowflake icon when frozen. Modules are filled in green when trained on common object images, in dark pink when trained on RS images, and with a gradient fill when trained on CO images and fine-tuned on RS images. Solid arrows represent the inference process, whereas dashed arrows indicate the "learning from" relationship.

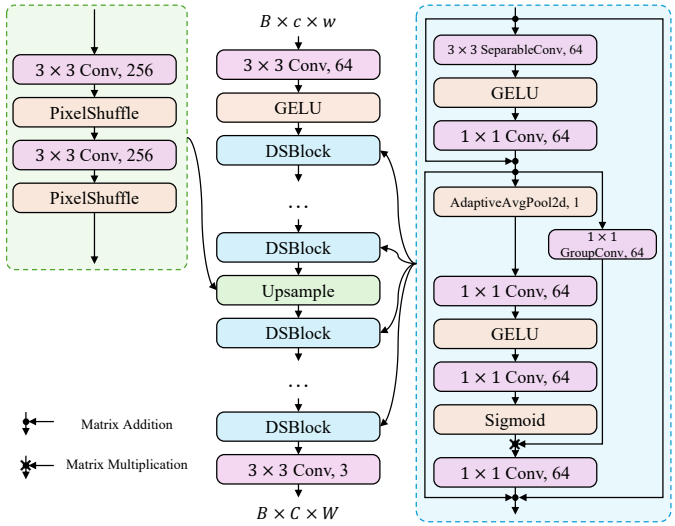


Fig. 2. Overall architecture of the proposed lightweight Tail model. **Left:** Detailed structure of the Upsample process in the Tail. **Middle:** The overall structure of the model. **Right:** Detailed architecture of the DSBlock used within the Tail.

The LR features are iteratively refined using a sequence of DSBlocks, formulated as

$$\mathbf{F}_{\text{LR}}^{i+1} = \text{DSBlock}(\mathbf{F}_{\text{LR}}^{i}), \quad i = 0, 1, \ldots, l-1, \tag{5}$$

where l is the number of DSBlocks applied in the LR space.

The refined LR features are then upsampled to the HR space using a sub-pixel convolution operation [1]:

$$\mathbf{F}_{\text{HR}}^{0} = \text{Upsample}(\mathbf{F}_{\text{LR}}^{l}), \tag{6}$$

and further refined iteratively in a manner analogous to the LR feature refinement, ultimately producing the final HR feature $\mathbf{F}_{\text{HR}}^{l}$.

Finally, the refined HR feature is decoded through a convolutional layer to generate the output feature map:

$$\mathbf{F}_{\text{out}} = \text{ConvDecoder}(\mathbf{F}_{\text{HR}}^{l}), \tag{7}$$

where ConvDecoder denotes the convolutional decoder layer.

Next, we provide further details of the DSBlock. The DSBlock is designed as an efficient feature refinement unit that integrates depthwise-separable convolution [2, 3] with an extremely lightweight EMA-style attention mechanism [4]. Given an input feature map $\mathbf{F} \in \mathbb{R}^{C\times h\times w}$, the DSBlock applies the following sequential transformation:

$$\mathbf{F}_{\text{T}} = \text{EMA}(\text{DSC}(\mathbf{F})), \tag{8}$$

where DSC denotes depthwise-separable group convolution, and EMA represents extreme memory attention.

The output of the block is computed through a residual connection with a scaling factor:

$$\mathbf{F}_{\text{out}} = \mathbf{F} + \alpha\mathbf{F}_{\text{T}}, \tag{9}$$

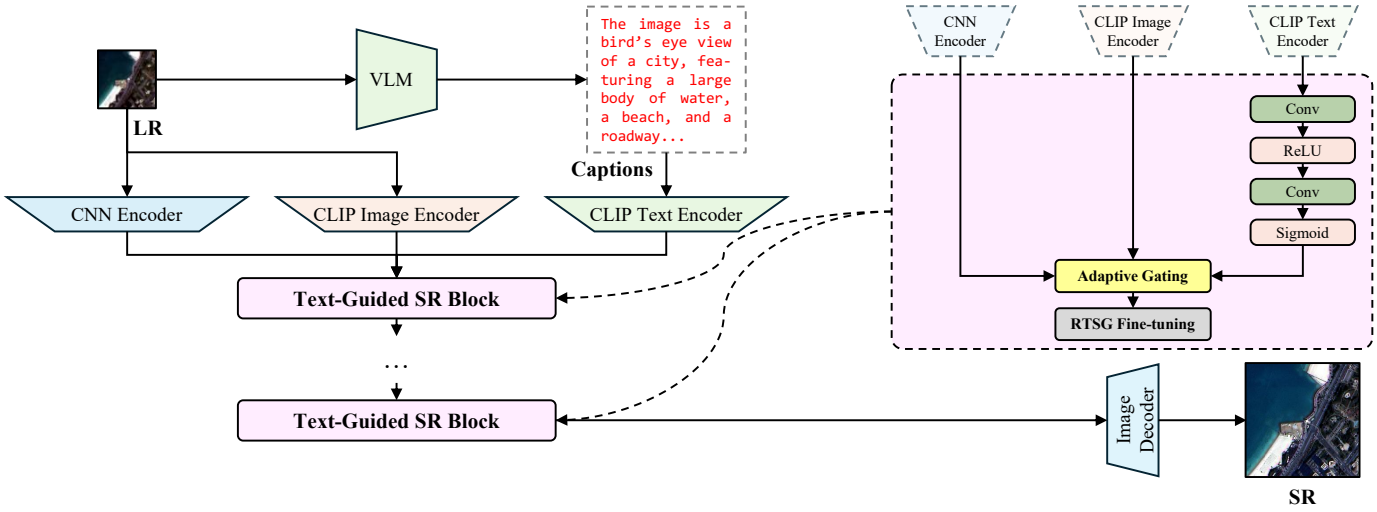


Fig. 3. Overall architecture of the proposed Tailless Text-Guided RSSR backbone. Modules marked with a fire icon are learnable during training, while those marked with a snowflake icon remain frozen.

where α is a learnable or fixed residual scaling factor.

In this stage, the Tail model is used to generate SR images. Let the LR input be $\mathbf{y} \in \mathbb{R}^{C\times h\times w}$, and the SR output of the SR Tail be $\text{SRTail}(\mathbf{y}) \in \mathbb{R}^{C\times H\times W}$. The objective of this stage is to train

$$\text{SRTail} = \arg\min \mathcal{L}_1(\mathbf{x}, \text{SRTail}(\mathbf{y})), \tag{10}$$

where $\mathbf{x} \in \mathbb{R}^{C\times H\times W}$ is the HR ground truth of the CO image, $\mathcal{L}_1$ denotes the L1 loss function [5], $s \in \mathbb{Z}_+$ is the scale factor, and $H = sh, W = sw$.

C. Stage 2: Learning Residual Correction via the Residual Tail

In this stage, with the SR Tail trained in the previous stage, another Tail (Residual Tail) is trained on the CO dataset, as shown in Figure 1b. The pretrained SR Tail is first used to generate Residuals for the CO dataset:

$$\mathbf{R} = \mathbf{x} - \text{SRTail}(\mathbf{y}), \tag{11}$$

where $\mathbf{x} \in \mathbb{R}^{C\times H\times W}, \mathbf{y} \in \mathbb{R}^{C\times h\times w}$ denote the HR ground truth and the LR image of the CO image, respectively, SRTail denotes the pretrained SR Tail model, and $\mathbf{R}$ denotes the Residual. The objective of this stage is to train

$$\text{ResTail} = \arg\min \mathcal{L}_1(\mathbf{R}, \text{ResTail}(\mathbf{y})). \tag{12}$$

Here, ResTail denotes the Residual Tail, $\mathcal{L}_1$ denotes the L1 loss function [5], $s \in \mathbb{Z}_+$ is the scale factor, and $H = sh, W = sw$.

D. Stage 3: Training the Tailless Text-Guided RSSR Backbone

In this stage, a Tailless Text-Guided RSSR backbone model is trained on the RS dataset. We introduce a multimodal RSSR pipeline in which text captions are incorporated as semantic guidance for super-resolution. The overall architecture of the proposed backbone is illustrated in Figure 3.

The model begins by processing the LR input through a pretrained VLM to generate descriptive captions, which serve as semantic cues for the RSSR task. To exploit multimodal information, we integrate multiple feature encoders, including a CNN Encoder, CLIP Image Encoder, and CLIP Text Encoder. Among them, only the CNN encoder is kept trainable while the others are frozen to limit the number of learnable parameters.

Similar to most RSSR frameworks, the proposed model employs an iterative refinement structure termed the Text-Guided SR Block, which progressively enhances SR features before decoding. Inside each block, text embeddings are first transformed by a multilayer perceptron (MLP), as shown on the right side of Figure 3. The resulting semantic vector is then combined with visual features via an Adaptive Gating module, enabling dynamic multimodal feature fusion. A fine-tuning module adapted from the RTSG structure in TTST [6] is finally applied to further refine the fused features before reconstruction.

We provide more details of the Adaptive Gating. Let the CNN Embeddings be $\mathbf{F}_{\text{CNN}}$, the CLIP Image Embeddings be $\mathbf{F}_{\text{CLIP}}$, and the processed Text Embeddings be $\text{MLP}(\mathbf{F}_{\text{Text}})$. Then the result of the Adaptive Gating can be described as

$$\begin{aligned}\mathbf{F}_{\text{Fuse}} &= \text{AdaGating}\big(\mathbf{F}_{\text{CNN}}, \mathbf{F}_{\text{CLIP}}, \text{MLP}(\mathbf{F}_{\text{Text}})\big) \\ &= (1-\lambda)\mathbf{F}_{\text{CNN}} \\ &\quad + \lambda\Big(\sigma\big(\text{MLP}(\mathbf{F}_{\text{Text}})\big) \odot \mathbf{F}_{\text{CNN}} + \mathcal{S}(\mathbf{F}_{\text{CLIP}})\Big),\end{aligned} \tag{13}$$

where λ is a learnable scalar gate for each Adaptive Gating module, $\sigma(\cdot)$ is the sigmoid nonlinearity, $\odot$ denotes element-wise (channel-wise, spatially broadcast) multiplication, and $\mathcal{S}(\mathbf{F}_{\text{CLIP}})$ denotes the semantic guidance maps produced from the global and local CLIP image embeddings via the SLM module and added as a bias term.

To obtain $\mathcal{S}(\mathbf{F}_{\text{CLIP}})$, the CLIP image features $\mathbf{F}_{\text{CLIP}}$ are first passed through several convolutional and upsampling operations to construct a spatial semantic prior. The process can be described mathematically as follows. Let $\mathbf{F}_{\text{CLIP}} \in \mathbb{R}^{C\times 1\times 1}$ be the global image embedding. It is first projected to the required

channel dimension through a linear mapping implemented by a 1×1 convolution:

$$\widetilde{\mathbf{F}}_{\text{CLIP}} = \phi\left(\text{Conv}_{1\times1}(\mathbf{F}_{\text{CLIP}})\right), \tag{14}$$

where $\phi(\cdot)$ denotes a nonlinear activation (*e.g.*, GELU). The projected embedding is then spatially broadcast to match the resolution of the current feature level

$$\widehat{\mathbf{F}}_{\text{CLIP}}(h, w) = \widetilde{\mathbf{F}}_{\text{CLIP}}, \quad \forall(h, w). \tag{15}$$

Finally, a sequence of convolutional refinements and upsampling operators $U(\cdot)$ are applied to generate structured spatial priors:

$$\mathcal{S}(\mathbf{F}_{\text{CLIP}}) = \psi\Big(U\big(\text{Conv}_{3\times3}(\widehat{\mathbf{F}}_{\text{CLIP}})\big)\Big), \tag{16}$$

where $\psi(\cdot)$ is a final activation used to produce a semantic response map bounded in $[0, 1]$. This mask injects global semantic knowledge from CLIP into the multimodal fusion process.

After an iterative refinement process through l Text-Guided SR Blocks, the features are passed to the Image Decoder, producing the final output of the RSSR backbone, as illustrated in Figure 3.

E. Stage 4: Residual Tail Fine-Tuning and Multi-Stage Tail Refinement for Final SR Output

In this stage, with the Tailless TG-RST trained in Stage 3, the Residual Tail trained in Stage 2 is fine-tuned on the RS dataset, as shown in Figure 1d. Let the pretrained Residual Tail be ResTail_0, and the Tailless TG-RST be SR_0. The objective of this stage is to fine-tune the ResTail to predict the Residual on RS dataset. In other words, to fine-tune

$$\text{ResTail} = \arg\min \mathcal{L}_1(\mathbf{x}, \text{SR}(\mathbf{y}) + \text{ResTail}(\mathbf{y})), \tag{17}$$

with initial values

$$\text{ResTail} = \text{ResTail}_0, \text{SR} = \text{SR}_0.$$

Here, $\mathbf{x} \in \mathbb{R}^{C\times H\times W}, \mathbf{y} \in \mathbb{R}^{C\times h\times w}$ denote the HR ground truth and the LR image of the CO image, respectively, $\mathcal{L}_1$ denotes the L1 loss function [5], $s \in \mathbb{Z}_+$ is the scale factor, and $H = sh, W = sw$.

The final output of the whole TG-RST is calculated as:

$$\hat{\mathbf{x}} = \text{SR}(\mathbf{y}) + \text{ResTail}(\mathbf{y}). \tag{18}$$

II. Experiments and Results

A. Overview

This study is implemented using PyTorch [7], with all experiments conducted on a system featuring an Intel Core i9-13900K CPU, 128 GiB of RAM, and a single NVIDIA GeForce RTX 4090 GPU. The operating system is Ubuntu 22.04 (Jammy) with CUDA version 12.4. For training the SR Tail model, we utilize two CO datasets: the ImageNet Large Scale Visual Recognition Challenge 2012 (ILSVRC2012) dataset [8] and the Places365 dataset [9]. Benchmark evaluations are performed on two public RSSR datasets: the Alsat-2B dataset [10] and the UC Merced Land Use Dataset [11].

TABLE I
Details on learnable parameters and FLOPs of the Tail models and different scales of Tailless TG-RST models.

Model	**Parameters** (M)	**FLOPs** (G)
SR Tail / Residual Tail	0.44	8.82
TG-RST_2 (Tailless)	10.49	44.12
TG-RST_3 (Tailless)	13.54	62.52
TG-RST_6 (Tailless)	22.72	117.69

The downscale factor s used to generate the LR images is set to 4. During training, random flipping and rotation are employed as data augmentation techniques [12], and mixed precision (via AMP) is used to reduce VRAM consumption [13]. For all datasets, we convert them into LMDB format to accelerate data loading and improve I/O efficiency [14]. The TG-RST framework (including all its stages), along with all baseline models used for comparison, employs L1 Loss [5] as the loss function and the AdamW optimizer [15] with an initial learning rate of 10^{-4}. A cosine learning rate decay strategy is applied during training. All models are trained from scratch, except for the CLIP components. In TG-RST, the VLM used to generate captions is LLaVa-1.5 7B [16]. The CLIP Text and Image Encoders used in the Text-Guided SR Blocks are pretrained and remain frozen during training. The initial aligning factor λ in the Text-Guided SR Block is set to 0.5. The RTSG Fine-Tuning unit is initialized based on the configuration provided in the original TTST paper [6]. To ensure fair comparisons with state-of-the-art methods, we scale the TG-RST model by adjusting the number of SR Units, l. We also add an Adaptive Gating module before decoding to create a similar model structure. Model performance is evaluated using Peak Signal-to-Noise Ratio (PSNR, computed on RGB channels) [17] and Structural Similarity Index Measure (SSIM) [18].

B. Comparing Methods

This study compares TG-RST's performance with state-of-the-art RSSR methods since 2024, including FAT [19], FMSR [20], GCRDN [21], TSFNet [22], and TTST [6]. The Frequency-Aware Transformer (FAT) [19] adopts a transformer backbone enhanced with a U-shaped attention fusion module and multiple layers of frequency-domain sparse self-attention to jointly capture global structure and frequency characteristics of remote sensing images. FMSR [20] builds a multi-level fusion architecture that integrates a Frequency Selection Module, a Vision State Space Module, and a Hybrid Gate Module to combine spatial and frequency-domain cues with efficient long-range modeling. GCRDN [21] adopts an encoder–decoder design where the encoder integrates nonlocal sparse attention into residual dense blocks to capture robust global context, and the decoder employs deep back-sampling blocks for iterative refinement. TSFNet [22] is a two-stage spatial–frequency joint learning framework that progressively refines SR results from coarse to fine by combining spatial-domain features with frequency-domain representations. TTST [6] is a transformer-based SR architecture featuring Residual Token Selective Groups, which dynamically select the top-k

TABLE II
QUANTITATIVE RESULTS FROM EXPERIMENTS ON TRAINING TAIL MODELS USING THE ILSVRC2012 AND PLACES365 DATASETS DEMONSTRATE THE EFFECTIVENESS OF TWO APPROACHES: SR TAILS, WHICH ARE TRAINED TO DIRECTLY PREDICT SR IMAGES, AND RESIDUAL TAILS, WHICH ARE TRAINED TO PREDICT THE RESIDUAL BETWEEN THE HR GROUND TRUTH AND THE SR IMAGES GENERATED BY THE SR TAILS.

Tail Model	Dataset	PSNR (dB)	SSIM
SR Tail	ILSVRC2012 [8]	23.9774	0.6604
	Places365 [9]	23.8481	0.6675
Residual Tail	ILSVRC2012 [8]	29.9704	0.6428
	Places365 [9]	29.7358	0.6360

most relevant tokens for compact and effective self-attention computation.

To ensure fair comparisons, we scale the TG-RST model by adjusting the number of SR Units, l, so that the compared methods have similar floating-point operations (FLOPs). Specifically, we use TG-RST with $l = 2$ (denoted as TG-RST$_2$) when comparing with FMSR and FAT, $l = 3$ (TG-RST$_3$) for comparisons with TSFNet and TTST, and $l = 6$ (TG-RST$_6$) when comparing with GCRDN. The learnable parameters and FLOPs of the full TG-RST model are computed as the sum of those from the Tailless TG-RST model and the corresponding Residual Tail. Details on learnable parameters and FLOPs are provided in Table I.

C. Pretrain Tail Models on Common Object Datasets

This study utilizes the ImageNet Large Scale Visual Recognition Challenge 2012 (ILSVRC2012) dataset [8] and the Places365 dataset [9] to train the Tail models. The ILSVRC2012 dataset contains 1,281,167 training images, 50,000 validation images, and 100,000 testing images. The Places365 dataset contains 1,803,064 training images, 328,500 validation images, and 36,500 testing images.

In our experiments, we only use the training and testing sets of both datasets. We obtain the Places365 dataset from Kaggle, which only contains 1,803,060 training images. To construct HR images, we first resize each ILSVRC2012 image such that the shorter edge is 224 px, and then center-crop a 224×224 region. As all images in Places365 are 256×256, we directly center-crop them to 224×224. We then downscale all HR images to 56×56 using PIL's resize function to obtain the LR counterparts.

In this study, we place four DSBlocks before and after the upsampling process within the Tail model, forming a lightweight and compute-efficient architecture with only 442.31 thousand learnable parameters and 8.82 giga FLOPs. We train the SR Tails on both datasets for 20 epochs from scratch, with an initial learning rate of 10^{-4}, a cosine decay schedule, and the AdamW optimizer [15]. We then evaluate the SR Tail on both splits to produce Residuals. Afterwards, we train the Residual Tails for one epoch from scratch, using the same optimization settings. We report the training results of the SR Tails and the Residual Tails in Table II. From the table we conclude that although with tight budget on parameters and FLOPs, the Tail models still can have a strong performance on super-resolving the LR image and learning the relation between the LR image and the Residual. Note that we intend not to train the Tails to convergence, in order to demonstrate the Tail model's performance in compute-constrained scenarios.

The results of training both SR Tails and Residual Tails on the ILSVRC2012 and Places365 datasets are presented in Table II. From the table, we observe that when trained to predict the SR image directly, the Tail models achieve PSNR values of approximately 24 dB and SSIM scores around 0.66, demonstrating strong performance in super-resolving images within a constrained budget of learnable parameters and FLOPs. When trained to predict the residual instead, the models attain PSNR values of around 30 dB and SSIM scores near 0.64, indicating that the lightweight Tail architecture effectively captures residual information. These results support the feasibility and efficiency of directly predicting residuals from LR images using a compact model.

The Residual Tails are subsequently fine-tuned and used to generate Residuals on RS datasets. To enhance the stability of the fine-tuning process, the outputs of the Residual Tails are downscaled by a factor of four prior to fine-tuning.

D. Comparison on Remote Sensing Datasets

In this study, we evaluate the methods' performance on two public RSSR datasets: the Alsat-2B dataset [10] and the UC Merced Land Use Dataset [11].

The Alsat-2B dataset, first presented by A. Djerida *et al.* [10], provides 2,182 samples for training and provides three distinct evaluation splits (agriculture, urban and special scenes), containing 56, 282, and 239 paired samples, respectively. The nominal ground sampling distance of the imagery is 2.5 m. A notable discrepancy in color characteristics exists between HR and LR data, which presents an additional challenge for RSSR. In this collection, HR patches are standardized to 256×256 pixels, whereas their LR counterparts measure 64×64 pixels.

The UC Merced Land Use Dataset, first presented by Y. Yang *et al.* [11], contains 21 land-use classes, each comprising 100 HR images. We randomly choose 75 samples per class for training, leaving 25 per class for evaluation. Every image is roughly 256×256 pixels with a ground sampling distance of 0.3 m.

For our experiments, we extract central crops of 224×224 as HR inputs and 56×56 for LR inputs. All networks are optimized over 200 epochs, with continuous tracking of PSNR on the held-out test data and an early-stopping patience of 40 epochs. The mini-batch size for TG-RST is fixed at 8. We fine-tune the pretrained Residual Tail for 5 epochs using the settings consistent with those described previously.

The quantitative results on both RS datasets are presented in Table III. For the TG-RST$_2$ which uses a Residual Tail pretrained on ILSVRC2012, the model achieves state-of-the-art performance on both datasets while maintaining strong SSIM scores of 0.3487 and 0.7721, respectively, surpassing other methods by 0.0565 dB in PSNR on Alsat-2B and 0.0779 dB in PSNR on UC Merced Land Use. TG-RST$_3$, despite

TABLE III
QUANTITATIVE RESULTS FROM EXPERIMENTS ON THE ALSAT-2B DATASET AND THE UC MERCED LAND USE DATASET ARE PRESENTED, WITH THE PERFORMANCE OF TG-RST MODELS SHOWN IN **BOLD**. FOR TG-RST MODELS, THE CO DATASET USED TO PRETRAIN THE TAIL MODELS IS PRESENTED IN THE BRACKET. THE BEST RESULT FOR EACH METRIC IS HIGHLIGHTED IN RED, AND THE SECOND-BEST RESULT IS MARKED IN BLUE.

Model	Parameters (M)	FLOPs (G)	Alsat-2B [10]		UC Merced Land Use [11]	
			PSNR (dB)	SSIM	PSNR (dB)	SSIM
FMSR [20]	7.05	47.02	16.2416	0.3500	27.2643	0.7697
FAT [19]	10.19	68.51	16.1959	0.3376	27.3023	0.7692
TG-RST$_2$ (ILSVRC2012)	**10.93**	**52.94**	**16.2971**	**0.3487**	**27.3802**	**0.7721**
TG-RST$_2$ (Places365)	**10.93**	**52.94**	**16.2941**	**0.3485**	**27.3797**	**0.7721**
TSFNet [22]	3.13	85.12	16.2394	0.3489	27.2989	0.7701
TTST [6]	18.30	117.66	16.1234	0.3540	27.4833	0.7759
TG-RST$_3$ (ILSVRC2012)	**13.98**	**71.34**	**16.3305**	**0.3516**	**27.4734**	**0.7769**
TG-RST$_3$ (Places365)	**13.98**	**71.34**	**16.3303**	**0.3517**	**27.4734**	**0.7769**
GCRDN [21]	31.53	679.57	16.1961	0.3412	27.4780	0.7748
TG-RST$_6$ (ILSVRC2012)	**23.16**	**126.51**	**16.3296**	**0.3488**	**27.5083**	**0.7783**
TG-RST$_6$ (Places365)	**23.16**	**126.51**	**16.3306**	**0.3494**	**27.5081**	**0.7783**

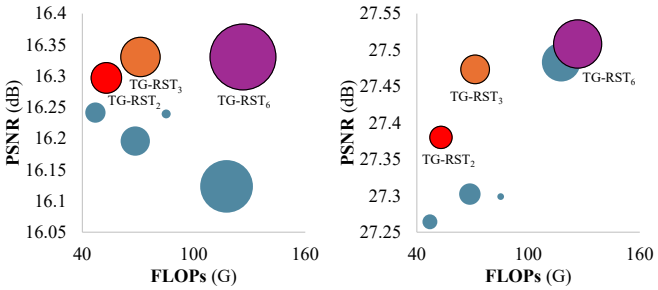


Fig. 4. Visualization of learnable parameter count, FLOPs, and PSNR for all methods except GCRDN on both RS datasets. Bubble size represents the number of learnable parameters. Methods closer to the top-left corner indicate better trade-offs between efficiency and performance. **Left:** Alsat-2B dataset. **Right:** UC Merced Land Use Dataset.

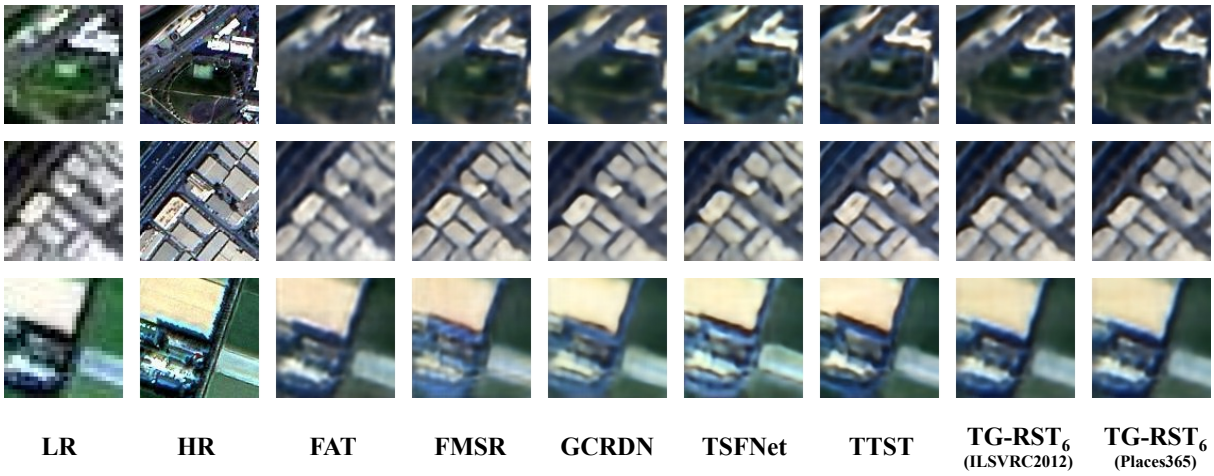
operating under the lowest FLOPs budget among the methods in the group, achieves the highest PSNR of 16.3305 dB on Alsat-2B and the highest SSIM of 0.7769 on UC Merced Land Use. It also ranks second-best in SSIM on Alsat-2B and in PSNR on UC Merced Land Use, slightly trailing TTST. TG-RST$_6$ attains the highest PSNR and SSIM scores across both datasets, outperforming all other methods. A visualization of these results is provided in the bubble chart shown in Figure 4. This chart highlights the trade-offs between performance and computational cost. Overall, TG-RST demonstrates strong performance under limited compute and achieves the best results when computation is unconstrained.

The qualitative results on both RS datasets are presented in Figure 5. From Figure 5a, we observe that TG-RST demonstrates superior domain adaptation capabilities. In the last row, unlike FMSR, GCRDN, and TSFNet, which show noticeable color hue distortions in the reconstructed images, TG-RST accurately restores the original color tones, particularly on the farm area (top-left corner) and the houses (bottom-left corner). TG-RST also exhibits enhanced ability to recover roads under significant downsampling. This is evident in the bottom part of the first-row image and the top-left corner of the second-row image. From Figure 5b, we conclude that TG-RST achieves better segmentation performance. For instance, in the first and second rows, TG-RST excels at reconstructing lines and repetitive patterns, such as the white lines in the second row. In the last row, TG-RST more effectively distinguishes cars from the background, indicating a higher-quality SR output.

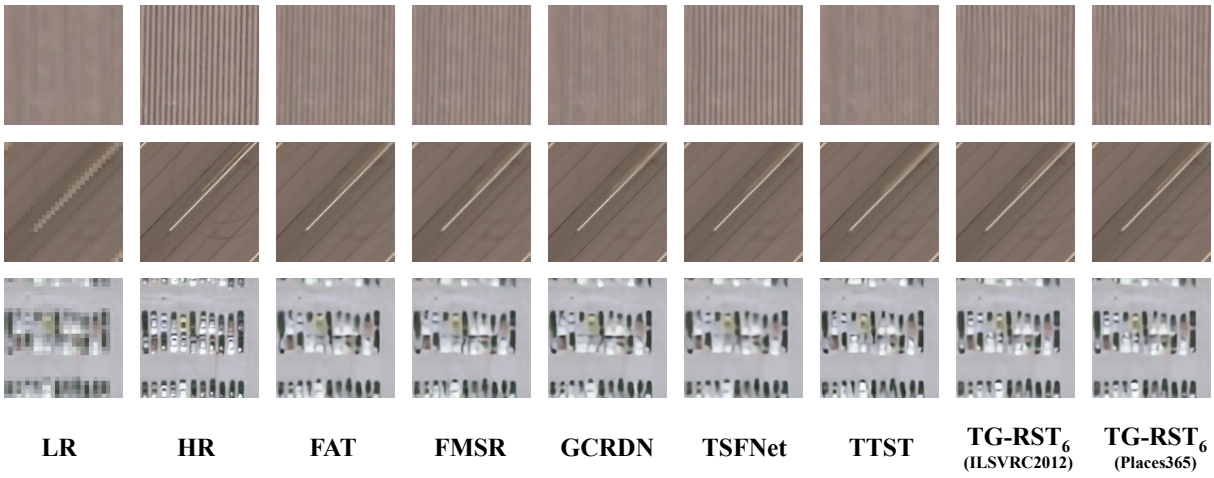
We provide visualization results on both RS datasets, as illustrated in Figure 6. From the figure, we observe that the generated text captions are generally accurate and beneficial for image reconstruction. In most cases, they provide meaningful semantic cues; however, in certain challenging scenarios—such as the second example from the Alsat-2B dataset—the captions may be inaccurate due to the high resolution required to correctly interpret the LR image. In that example, the VLM misidentifies the LR image as "a city street at night," which is incorrect. Regarding the Semantic Response Maps in the Text-Guided SR Blocks, we note that their focus shifts across different stages of the blocks. While their emphasis varies, certain maps consistently capture semantics aligned with the content highlighted in the text, demonstrating their utility in guiding the refinement process. In terms of Residuals, we find that most SR models struggle to reconstruct high-frequency details, especially along edges. This issue is more pronounced in the UC Merced Land Use Dataset, as shown in Figure 6b, likely because this dataset contains more structured elements and sharp boundaries than Alsat-2B, making edge details more critical. This observation is also supported by the comparison between the super-resolved images and the HR ground truths in Figure 5. The Predicted Residual captures much of this missing edge information, but it also includes some low-frequency content—for instance, the yellow earth region in the first example from the Alsat-2B dataset. Overall, the Residual Tail effectively learns and restores edge details absent from the baseline SR output, thereby enhancing the final SR performance.

E. Ablation Study

In this study, we conduct an ablation experiment on TG-RST following this steps. We first build a baseline model by removing the CLIP Image Encoder and the CLIP Text Encoder in the Text-Guided SR Block and the Residual Tail. Then we

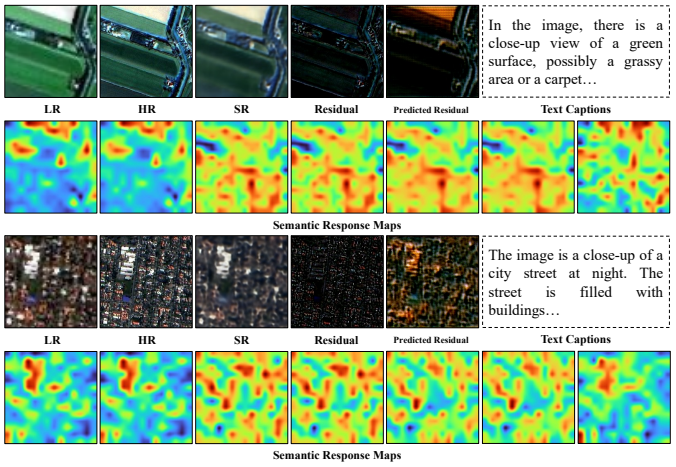


(a) Visualization of comparisons on Alsat-2B dataset.

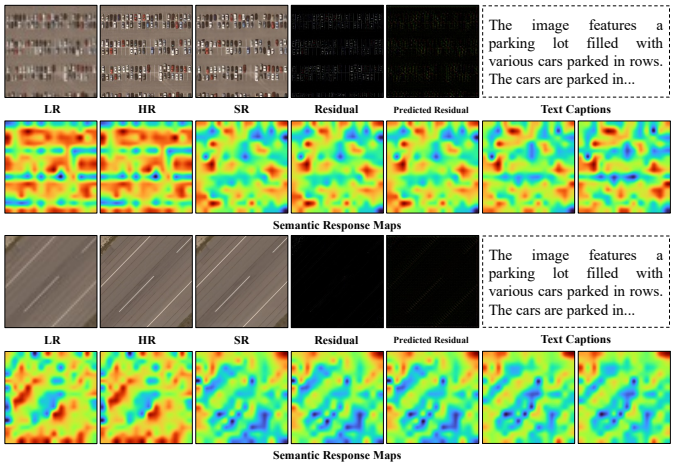


(b) Visualization of comparisons on UC Merced Land Use Dataset.

Fig. 5. Visualization of comparisons on both the Alsat-2B dataset and the UC Merced Land Use Dataset. For TG-RST, only the results with $l = 6$ are shown. The dataset indicated in brackets refers to the CO dataset used to pretrain the Residual Tail of TG-RST.



(a) Visualization of comparisons on Alsat-2B dataset.



(b) Visualization of comparisons on UC Merced Land Use Dataset.

Fig. 6. Visualization results of selected examples from TG-RST ($l = 6$) outputs on both the Alsat-2B dataset and the UC Merced Land Use Dataset. For Alsat-2B, the Residual Tail is pretrained on Places365, and for UC Merced Land Use, it is pretrained on ILSVRC2012. The "Residual" image represents the difference between the output of the Tailless TG-RST model and the HR ground truth, while the "Predicted Residual" image shows the output generated by the Residual Tail. For the UC Merced Land Use Dataset, please zoom in on the "Residual" and "Predicted Residual" images for a clearer view of fine details.

TABLE IV
QUANTITATIVE RESULTS FROM THE ABLATION STUDY ON BOTH DATASETS. IN THE "MODEL" COLUMN, "TE" REFERS TO THE CLIP TEXT ENCODER AND "IE" REFERS TO THE CLIP IMAGE ENCODER. THE BEST RESULTS ARE HIGHLIGHTED IN **BOLD**, AND THE SECOND-BEST RESULTS ARE UNDERLINED.

Model	Alsat-2B [10]		UC Merced Land Use [11]	
	PSNR (dB)	**SSIM**	**PSNR** (dB)	**SSIM**
Base.	16.1801	0.3298	27.4939	0.7766
Base. + TE	16.1982	0.3527	27.5011	0.7775
Base. + TE + IE	16.3218	**0.3533**	27.5036	**0.7783**
Full	**16.3306**	0.3494	**27.5083**	**0.7783**

TABLE V
QUANTITATIVE RESULTS FROM EXPERIMENTS ON TRAINING TAIL MODELS USING THE ILSVRC2012 AND PLACES365 DATASETS DEMONSTRATE THE EFFECTIVENESS OF TWO APPROACHES: SR TAILS, WHICH ARE TRAINED TO DIRECTLY PREDICT SR IMAGES, AND RESIDUAL TAILS, WHICH ARE TRAINED TO PREDICT THE RESIDUAL BETWEEN THE HR GROUND TRUTH AND THE SR IMAGES GENERATED BY THE SR TAILS. THE BEST RESULTS ARE HIGHLIGHTED IN **BOLD**, AND THE SECOND-BEST RESULTS ARE UNDERLINED.

Tail Model	Alsat-2B [10]		UC Merced Land Use [11]	
	PSNR (dB)	**SSIM**	**PSNR** (dB)	**SSIM**
Tailless	16.3218	**0.3533**	27.5036	**0.7783**
From Scratch	16.3276	0.3488	27.5078	**0.7783**
SR, ILSVRC2012	16.3164	0.3491	27.4997	0.7778
SR, Places365	16.3101	0.3455	27.5074	**0.7783**
Better Residual	**16.3306**	0.3494	**27.5083**	**0.7783**

add add the CLIP Text Encoder. After that we add the CLIP Image Encoder. At last we add the Residual Tail.

The quantitative results are presented in Table IV. From the table, we observe that each component (CLIP Text Encoder, CLIP Image Encoder, and the Residual Tail) contributes to performance improvement. Notably, incorporating text captions significantly enhances SSIM on the Alsat-2B dataset by 0.0229 and boosts PSNR on the UC Merced Land Use Dataset by 0.0072 dB. This may be attributed to the dataset characteristics: Alsat-2B contains only three remote sensing categories, while UC Merced Land Use encompasses a broader range of 21 categories. As a result, the semantic diversity in UC Merced Land Use allows the model to benefit more from the additional contextual information provided by text captions. Furthermore, given the evident domain shift between LR and HR images in the Alsat-2B dataset, the inclusion of CLIP Image Embeddings offers valuable information from the LR domain, helping the model better adapt to this shift. Finally, integrating the Residual Tail with the baseline model yields consistent performance gains on both datasets, reinforcing the effectiveness of the proposed Tail architecture.

F. Multi-Stage Training Pipeline Study

In this study, we conduct additional experiments to further evaluate the design of the multi-stage Tail training pipeline. First, we train the Residual Tail model from scratch, omitting pretraining on CO datasets. Next, we explore initializing the Residual Tail directly from a pretrained SR Tail, leveraging potential encoder/decoder knowledge transfer. Tailless TG-RST$_6$ is used as the baseline SR method, and all experiments are performed on the UC Merced Land Use Dataset. The results of these experiments are reported in Table V. From the table, we observe that adding a Residual Tail to TG-RST leads to PSNR gains of 0.0092 dB on the Alsat-2B dataset and 0.0047 dB on the UC Merced Land Use Dataset. Training the Residual Tail from scratch also yields moderate improvements, with PSNR gains of 0.0058 dB on Alsat-2B and 0.0042 dB on UC Merced Land Use. In contrast, fine-tuning the Residual Tail from a pretrained SR Tail results in poorer performance, likely because extracting residuals from LR inputs requires a different feature representation than that used for direct super-resolution. Additionally, we observe a larger performance gap between scratch and pretrained Residual Tails on Alsat-2B compared to UC Merced Land Use. Similarly, the performance drop when fine-tuning from the SR Tail is more pronounced on Alsat-2B. These observations suggest that the proposed multi-stage Tail training strategy is particularly beneficial when image reconstruction is more challenging, potentially due to larger downscaling factors, domain shifts between LR and HR images, or greater visual complexity.

We also attach Residual Tails to other methods, with the experimental results summarized in Table VI. From the table, we conclude that all methods benefit from the addition of the Residual Tail on the UC Merced Land Use Dataset, while FMSR, TSFNet, and TTST also show performance gains on the Alsat-2B dataset. However, we observe a degradation in SSIM performance on Alsat-2B. This may be attributed to the domain shift between the HR and LR images in the Alsat-2B dataset, which makes the Residuals more challenging to predict accurately. As a result, the effectiveness of the Residual Tail is reduced, leading to lower SSIM values compared to its performance on the UC Merced Land Use Dataset.

III. CONCLUSION

This paper presented TG-RST, a text-guided remote sensing super-resolution framework that couples a tailless multimodal RSSR backbone with a lightweight, multi-stage Tail refinement strategy. TG-RST leverages VLM-generated captions and frozen CLIP encoders to inject semantic priors into iterative SR feature refinement via Adaptive Gating and semantic response maps, while keeping the number of trainable parameters controlled through a CNN-centric design. In parallel, we exploit the empirical regularity of SR residuals by introducing a compact Tail architecture built from depthwise-separable blocks with efficient attention, and by structuring training into four stages that pretrain an SR Tail and a Residual Tail on large-scale common-object datasets before fine-tuning the Residual Tail for remote sensing residual correction. Across Alsat-2B and UC Merced Land Use benchmarks under a $\times 4$ setting, TG-RST consistently achieves state-of-the-art or best-in-class PSNR/SSIM under matched FLOPs budgets, and qualitative comparisons further show improved restoration of color fidelity, roads, structured patterns, and edge details relative to recent transformer- and frequency-based baselines.

Ablation and pipeline studies further validate that the gains stem from complementary contributions of text guidance,

TABLE VI
QUANTITATIVE RESULTS FROM EXPERIMENTS ON ATTACHING TAIL MODELS TO OTHER STATE-OF-THE-ART METHODS. THE DATASET NAME IN BRACKETS IN THE "TAIL MODEL" COLUMN INDICATES THE CO DATASET USED TO TRAIN THE RESIDUAL TAIL. FOR EACH METHOD, THE BEST RESULT IS HIGHLIGHTED IN **BOLD**.

Baseline SR Model	Tail Model	Parameters (M)	FLOPs (G)	Alsat-2B [10]		UC Merced Land Use [11]	
				PSNR (dB)	SSIM	PSNR (dB)	SSIM
FAT [19]		10.19	68.51	**16.1959**	**0.3376**	27.3023	0.7692
	+ (ILSVRC2012)	10.63	77.33	16.1929	0.3356	**27.3038**	**0.7693**
	+ (Places365)	10.63	77.33	16.1904	0.3366	**27.3038**	**0.7693**
FMSR [20]		7.05	47.02	16.2416	**0.3500**	27.2643	0.7697
	+ (ILSVRC2012)	7.49	55.84	**16.2476**	0.3483	**27.2828**	**0.7698**
	+ (Places365)	7.49	55.84	16.2453	0.3483	27.2827	0.7697
GCRDN [21]		31.53	679.57	**16.1961**	**0.3412**	27.4780	0.7748
	+ (ILSVRC2012)	31.97	688.39	16.1748	0.3316	**27.4876**	**0.7751**
	+ (Places365)	31.97	688.39	16.1751	0.3334	27.4874	**0.7751**
TSFNet [22]		3.13	85.12	16.2394	**0.3489**	27.2989	**0.7701**
	+ (ILSVRC2012)	3.57	93.94	**16.2395**	0.3446	**27.3004**	**0.7701**
	+ (Places365)	3.57	93.94	16.2376	0.3419	27.3000	**0.7701**
TTST [6]		18.30	117.66	16.1234	**0.3540**	27.4833	0.7759
	+ (ILSVRC2012)	18.74	126.48	**16.1469**	0.3532	**27.4877**	**0.7760**
	+ (Places365)	18.74	126.48	16.1468	0.3537	27.4876	0.7759

CLIP-based visual semantics, and residual correction: captions notably improve structural consistency (especially SSIM on Alsat-2B), CLIP image embeddings support robustness under LR–HR domain shift, and the Residual Tail provides consistent PSNR improvements by recovering high-frequency content missing from the backbone output. Experiments also indicate that the proposed residual-pretraining-and-fine-tuning paradigm is preferable to training a Residual Tail from scratch and substantially more effective than initializing residual prediction from an SR Tail, underscoring that residual inference requires distinct representations. Finally, attaching the Residual Tail to multiple state-of-the-art RSSR models yields additional improvements (particularly on UC Merced) suggesting that the Tail operates as a generally applicable refinement module. Future work may focus on improving caption reliability under extreme downsampling and further mitigating LR–HR domain shifts (*e.g.*, through stronger semantic priors or domain-adaptive residual modeling), to increase the stability of residual refinement in challenging remote sensing scenarios.

REFERENCES

[1] W. Shi, J. Caballero, F. Huszár, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert, and Z. Wang, "Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 2016, pp. 1874–1883.

[2] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1251–1258.

[3] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," in *arXiv preprint arXiv:1704.04861*, 2017.

[4] Q. Wang, B. Wu, P. Zhu, P. Li, Q. Hu, and W. Li, "Ecanet: Efficient channel attention for deep convolutional neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11 534–11 542.

[5] H. Zhao, O. Gallo, I. Frosio, and J. Kautz, "Loss functions for image restoration with neural networks," *IEEE Transactions on Computational Imaging*, vol. 3, no. 1, pp. 47–57, 2017.

[6] Y. Xiao, Q. Yuan, K. Jiang, J. He, C. W. Lin, and L. Zhang, "Ttst: A top-k token selective transformer for efficient self-attention," *IEEE Transactions on Image Processing*, vol. 33, pp. 738–752, 2024.

[7] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," 2019.

[8] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, "ImageNet Large Scale Visual Recognition Challenge," *International Journal of Computer Vision (IJCV)*, vol. 115, no. 3, pp. 211–252, 2015.

[9] B. Zhou, A. Lapedriza, A. Khosla, A. Oliva, and A. Torralba, "Places: A 10 million image database for scene recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 6, pp. 1452–1464, 2018.

[10] A. Djerida, K. Djerriri, M. Karoui, and M. Larabi, "A new public alsat-2b dataset for single-image super-resolution."

[11] Y. Yang and S. Newsam, "Bag-of-visual-words and spatial extensions for land-use classification," in *ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (ACM GIS)*, 2010.

[12] A. Mumuni, F. Mumuni, and N. K. Gerrar, "A survey of synthetic data augmentation methods in machine vision," *Machine Intelligence Research*, vol. 21, no. 5, p. 831–869, Mar. 2024.

[13] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," 2018.

[14] S. Pumma, M. Si, W.-C. Feng, and P. Balaji, "Parallel i/o optimizations for scalable deep learning," in *Proceedings of the IEEE International Conference on Parallel and Distributed Systems (ICPADS)*, Shenzhen, China, Dec 2017, pp. 720–729.

[15] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," *International Conference on Learning Representations*

(ICLR), 2015.
[16] H. Liu, C. Li, Y. Li, and Y. Lee, "Improved baselines with visual instruction tuning (llava-1.5)," *arXiv preprint*, 2024.
[17] A. N. Netravali and B. G. Haskell, "Digital pictures: Representation and compression," in *Springer*, 1988.
[18] Z. Wang, A. Bovik, H. Sheikh, and E. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
[19] X. Yan, J. Chen, Q. Xu, and W. Li, "Mitigating texture bias: A remote sensing super-resolution method focusing on high-frequency texture reconstruction," *IEEE Transactions on Geoscience and Remote Sensing*, 2025.
[20] Y. Xiao, Q. Yuan, K. Jiang, Y. Chen, Q. Zhang, and C.-W. Lin, "Frequency-assisted mamba for remote sensing image super-resolution," *IEEE Transactions on Multimedia*, vol. 26, pp. 1–14, 2024.
[21] J. Sui, X. Ma, X. Zhang, and M.-O. Pun, "Global context-driven residual dense network for remote sensing image super-resolution," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 16, pp. 4457–4468, 2023.
[22] J. Wang, Y. Lu, S. Wang, B. Wang, X. Wang, and T. Long, "Two-stage spatial-frequency joint learning for large-factor remote sensing image super-resolution," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 62, pp. 1–14, 2024.